BirdCLEF 2026: An LLM-Guided Agentic Framework for Passive Acoustic Monitoring

Advanced Predictive Analytics 2025/2026, Project Report

Enrico Zafiris · Lasse Aaron Ortmann · Moritz Üblacker · Alexander Wanstrath

Abstract.

We present a staged autonomous research agent for the BirdCLEF 2026 Kaggle competition, a 234-class multi-label bird species classification task evaluated on macro-averaged ROC-AUC over passive soundscape recordings of the Pantanal wetlands. Our approach replaces manual hyperparameter search with a four-role LLM-guided loop (Researcher, Coder, Executor, Evaluator) that iteratively proposes, implements, evaluates, and refines candidate architectures under a persistent JSONL experiment memory. Two agent-driven modelling tracks, a CNN trained from scratch on log-mel spectrograms and a trainable head on frozen Google Perch embeddings, compete in a cheap tournament on approximately 1,500 training samples before committing to full-data training. BirdNET v3.1 was evaluated as a third encoder but excluded from the autonomous pipeline in favour of Perch, which demonstrated superior acoustic representations for this task. A critical finding is that macro-averaged ROC-AUC systematically misranks models locally relative to the Kaggle leaderboard; switching to macro average precision (macro-AP) as the primary optimisation target corrected this inversion. Starting from a naïve convolutional baseline of approximately 0.65 macro-AUC, our autonomous agent reached a final public leaderboard score of 0.88 across 22 submission versions and more than 215 tracked agent iterations, representing a 38-percentage-point improvement driven principally by domain-specific pre-trained acoustic representations and smart metric selections. Our autonomous agent outperformed every manually constructed baseline.

1. Introduction

Passive acoustic monitoring (PAM) enables scalable, non-invasive biodiversity assessment via autonomous recording units that capture continuous ambient audio. The BirdCLEF 2026 Kaggle competition operationalises PAM at scale: classify vocalisations of 234 bird species from Pantanal soundscape recordings, evaluated on macro-averaged ROC-AUC. Three difficulties make naïve approaches insufficient: (1) **multi-label** outputs per five-second clip; (2) a **severe long-tail** class distribution with some species having even fewer than 10 examples; (3) a **focal-to-soundscape domain gap** between clean training recordings and noisy ambient test data. Rather than committing to a single architecture, we designed a **staged autonomous** research agent progressing from ≈0.65 to **0.88** macro-AUC (+38 pp). **Agent summary**: A four-role LLM loop drives all experimental decisions: a **Researcher** proposes configurations, a **Coder** implements them, an **Executing-Pipeline** runs training in isolated subprocesses, and an **Evaluator-Script** scores the results. Outcomes are written to an append-only JSONL memory, guiding the loop from initial data exploration through to the final Kaggle submission files.

2. Dataset & Exploratory Data Analysis

2.1 Dataset Overview & EDA Agent Findings

The training set covers 234 species native to or migratory through the Pantanal. Each recording carries a primary species label and optionally secondary-species labels. The test set consists of 60-second soundscape recordings from passive sensors, segmented into twelve non-overlapping five-second windows for model inference.

Before any model training, a dedicated EDA agent analysed the dataset and produced a concise report that was injected into all Researcher LLM prompts. Key findings:

- **Class imbalance:** The top 20 species account for the majority of labelled clips; many of the 234 species have fewer than 20 training examples. This directly motivated class-frequency-weighted loss functions throughout.
- **Duration variability:** Focal training clips vary in duration; clips shorter than five seconds are zero-padded, while longer clips are truncated to the first five seconds.
- **Sampling rate variability:** All training audio was resampled to 32,000 Hz for consistent mel-spectrogram computation.
- **Focal vs. soundscape structure:** Training recordings are predominantly focal; the available labelled train_soundscapes are the only in-distribution proxy for the test set. They were reserved exclusively for evaluation during architecture and augmentation search (Phase 2); Phase 1e fine-tuning additionally incorporated them as supervised training data (sample_weight 1.0).

- **Secondary label reliability:** Secondary species annotations are sparse, present in only 12.3% of recordings, and were therefore excluded as primary training targets.

The EDA output was deliberately kept as a single compact text file rather than a collection of plots, so it could be injected directly into LLM prompts without consuming excessive context budget; a design principle applied throughout the pipeline.

2.3 Audio Preprocessing Pipeline

All recordings are unified to a common representation in four steps: (1) Resampling to 32,000 Hz using high-quality polyphase resampling; (2) Five-second chunking into non-overlapping windows, with zero-padding for short clips; (3) Mel spectrogram computation using 64 mel filter banks and 128 time frames per clip, converted to dB scale via $10 \times \log_{10}(S / S_max)$ to compress dynamic range; (4) No further normalisation is applied after dB conversion. The resulting input tensor shape is 64×128×1. This configuration was empirically validated against the alternative (128 mels/256 frames) and kept fixed across all model families so that metric differences reflect architectural rather than preprocessing choices.

2.4 Online Data Augmentation

We defined several augmentation presets ranging from no augmentation to a high-intensity configuration and an extreme stress-test variant. The high preset ended up as a fixed baseline during architecture search, ensuring that all candidate architectures are evaluated under strong domain-adaptation conditions. We decided to implement a high baseline because of the gap between focal audio recordings (training) and real jungle audio (testing). Within Phase 1 (CNN) and Phase 2 (Perch), once the best architecture is locked, the agent performs a dedicated augmentation search - exploring a diverse range of LLM-generated strategies, with the high preset as anchor. Using this strategy, our agent was able to cheaply identify promising models without the need to constantly adapt the augmentation search.

3. System Architecture & Agent Design

3.1 Why a Staged Pipeline, not a Monolithic Agent

An initial attempt to build a single monolithic agent failed for three interconnected reasons: (1) models available for local LLM inference (7-14B-parameter coders) lack long-horizon planning capacity; (2) without executable templates and frozen components, generated code broke frequently via syntax errors, shape mismatches, and missing imports; (3) without a leaderboard-aligned metric, the agent's feedback drove it toward locally-good but globally-weak architectures.

The resolution was a manual notebook phase before building the agent, which served three critical functions: establishing sensible score ranges for each model family; discovering that macro-averaged ROC-AUC (the competition's official test metric) mis-ranks models locally and adopting macro average precision (macro-AP) as the internal agent ranking metric; And constructing the environment tooling needed to give our agent a safe, structured framework for optimal self-directed learning.

3.2 End-to-End Pipeline Stages

- Phase 0 (EDA Agent): Explores the dataset programmatically and produces a summary, which is injected as shared context into Researcher prompts across all tracks.
- Phase 1 (CNN Track): The agent explores CNN architectures and hyperparameter combinations on ≈1,500 samples (3 epochs), with augmentation fixed at the high preset (for the first exploration phase). A Researcher → Coder → Executor → Evaluator loop runs for an adjustable number of iterations, progressively refining the champion architecture. The augmentation search then identifies and locks Time Mask, Frequency Mask, and Noise as the optimal configuration.
- The agent searches head architectures over frozen 1,536-dim ONNX embeddings on the same ≈1,500-sample budget. The same four-role loop applies; augmentation search adjusts SNR-Mixing, Noise, Time Shift, Pitch Shift, and Time Stretch. Beyond feature extraction, Perch's internal vocabulary (~14,000 species) is leveraged via direct logit mapping and genus-level proxies for species absent from its vocabulary, enabling zero-shot blending at inference.
- Phase 3 (BirdNet Track - Disabled): Pretrained BirdNet embeddings (1,024-dim) were evaluated, in earlier versions of our agent, however the track is now excluded from the autonomous tournament and final code (replaced by Perch).
- Phase 4 (Tournament): CNN and Perch track champions are evaluated on the soundscape validation set (the only real test data that we have) using macro-AP; the winner's specification is taken to the next phase, the losing track is discarded.

- Phase 5 (Full-Data Retrain): Tournament winner retrained on whole dataset with the winning augmentation and architecture.
- Phase 6 (Pseudo-Label Fine-Tune): ≈10,000 unlabeled soundscapes pseudo-labelled with fixed thresholds (top-1 ≥ 0.55, runner-up ≤ 0.35, label weight 0.5); final model fine-tuned on focal + labelled soundscapes + pseudo-labelled data, producing a Kaggle-ready submission.

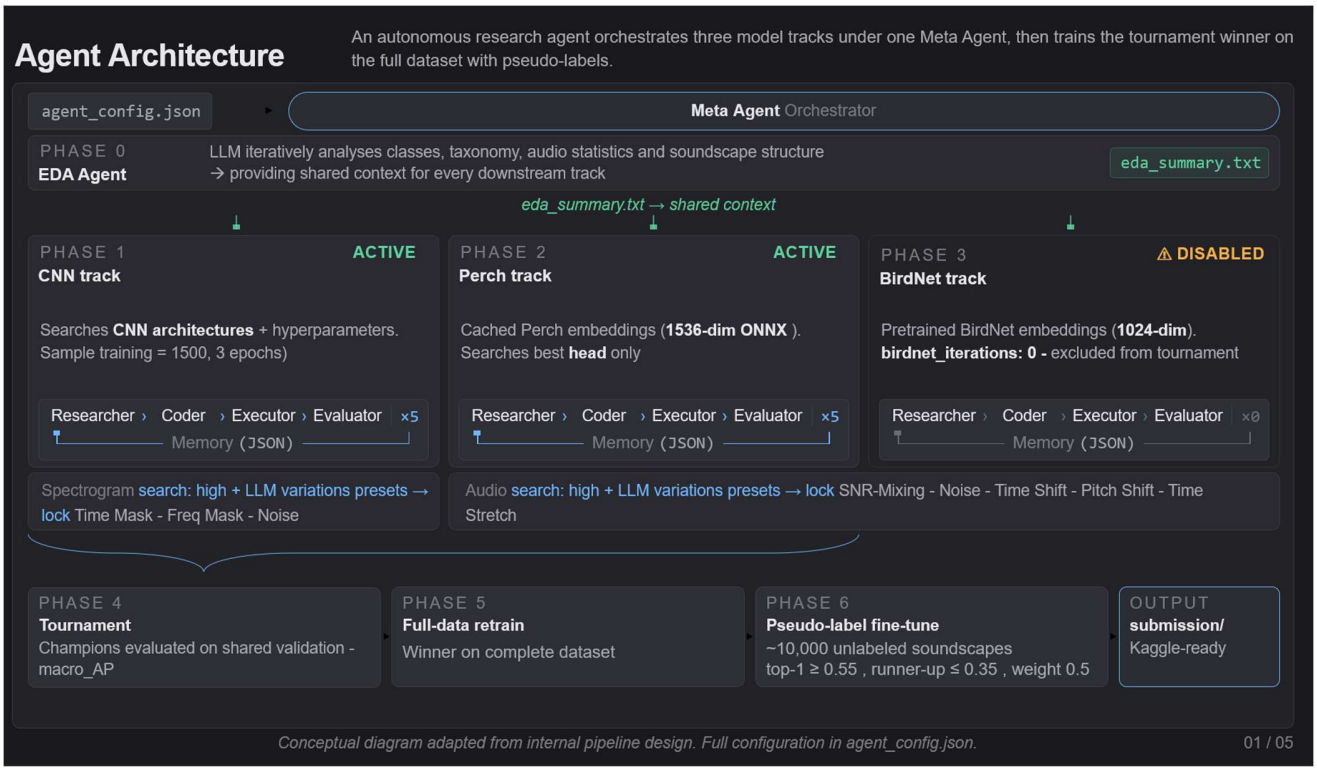


Figure 1: Overview of the Agent Architecture

3.3 LLM-Involvement in different Phases

Stage	Researcher (LLM - proposes)	Coder (LLM - implements)	Fixed by designer (not LLM-chosen)
Phase 0 – EDA Agent	Proposes EDA queries and dataset exploration steps (class distribution, duration statistics, sampling rate variability, focal vs. soundscape structure).	Translates exploration steps into executable Python. Writes results to eda brief.txt.	Output format (single compact text file). Max length (600 chars) for LLM context injection.
Phase 1 – Architecture Search	Proposes new model architectures as JSON (CNN: Conv2D stacks, pooling, classifier head, LR, dropout, BN; Perch: head type, hidden_dim, dropout, proj_dim). Reads experiment memory to avoid repetition and guide decisions.	Translates each proposal into runnable Python. Auto-fixes runtime errors (up to 5 retries).	Augmentation preset (high). Eval protocol (macro-AP on soundscapes). Data budget (≤1,500 focal clips). Perch ONNX encoder frozen.
Phase 2 – Architecture Refinement	Proposes small modifications around the Stage 1a champion (same architecture family, exploit strategy): LR, dropout, layer sizes, weight decay. No new architectures.	Writes/updates code within the locked architecture family. Same train→eval loop as Stage 1a.	Architecture family locked (e.g. residual_cnn / residual_mlp). Iteration budget: initial 5 + up to 2 bonus runs for improvement. However, these budgets are adjustable via our config file.
Phase 3 – Augmentation Search	Proposes augmentation parameters as JSON only (SNR-Mixing, BG-Mix, Time-Stretch, Pitch-Shift, Noise, Gain-Jitter). Does not redesign head or architecture.	Perch: head code frozen; Coder re-embeds audio and retrains fixed head per aug trial. CNN: architecture frozen; Coder invoked mainly to repair the locked slot-script.	Fixed clip subset for fair comparison and a preset baseline (high) as an anchor run without LLM.
Phase 4 – Tournament	*No LLM involvement.*	*No LLM involvement. Deterministic comparison: CNN champion vs. Perch*	Ranking metric: macro-AP. Loser discarded; winner specification gets into the next phase.

Stage	Researcher (LLM - proposes)	Coder (LLM - implements)	Fixed by designer (not LLM-chosen)
		champion on shared soundscape validation set.	
Phase 5 – Full-Data Retrain	*No LLM involvement.*	*No LLM involvement. Deterministic training on all focal data. Automatic soundscape scoring after training.*	Architecture + aug config from tournament winner. val_split=0.0 (all data for training).
Phase 6 – Pseudo-Label Fine-Tune	*No LLM involvement.*	*No LLM involvement. Phase 5 model as teacher → pseudo-labels for unlabelled soundscapes. Fine-tune on supervised + pseudo-data (5 epochs, LR=2e-4, supervised weight 1.0, pseudo weight 0.5).*	Fixed thresholds: top-1 ≥0.55, runner-up <0.35, label weight 0.5. Teacher weights from Phase 5. Output: model_pseudo.keras / final_head_pseudo.keras.

Table 1: Stage-by-stage LLM involvement and fixed parameters per pipeline stage. Phase 5 and Phase 6 are fully deterministic; LLM-guided decisions are confined to stages Phase 0-3.

3.4 Experiment Memory Design

The ExperimentMemory module is an append-only JSONL file (one JSON object per line per experiment) stored in a per-stage directory. This design is crash-safe, human-readable, and supports both sequential reading and filtered random access. Each record stores: the full experiment specification; a success/failure flag with error reason; all evaluation metrics (macro-AP, macro-AUC, median per-class AUC); the Researcher's stated reasoning and hypothesis; and a timestamp. A separate experiment timeline.Json aggregates all stages for visualisation.

The Researcher prompt is structured as: (1) EDA brief (max 600 chars of dataset insights); (2) the single best prior result by macro AP plus all prior runs in chronological order; (3) 5 most recent failure cases with error reasons; (4) the allowed parameter space with all valid values, embedded in the system prompt; (5) required output format (JSON only, no prose). The Coder receives only the current JSON spec and writes a complete executable training script. CNN and Perch agents maintain independent memories, sharing only the EDA brief. This separation is intentional: the two tracks use fundamentally different architectures and training regimes, so experiment history from one would not provide useful signal to the other's Researcher LLM.

The Researcher prompt is structured as: (1) EDA brief (max 600 chars of dataset insights); (2) top-k prior results sorted by macro-AP; (3) 5 most recent failure cases with error types; (4) metric trend over recent iterations; (5) allowed JSON schema with all valid parameter values; (6) required output format (JSON only, no prose). The Coder receives only the current JSON spec and writes a complete executable training script. CNN and Perch agents maintain independent memories, sharing only the EDA brief. This separation is intentional: the two tracks use fundamentally different architectures and training regimes, so experiment history from one would not provide useful signal to the other's Researcher LLM.

3.5 Search Budget and Augmentation Strategy

All architecture and augmentation search runs on a fixed subset of approximately 1,500 training samples, grounded in neural scaling law observations: the relative ranking of architectural choices is preserved as data and compute scale up. This budget represents the minimum for a stable local evaluation signal, while keeping per-experiment wall-clock time bounded. Full-data training is reserved exclusively for the single tournament winner. Augmentation is fixed at the high preset throughout architecture search phases (Phases 1 and 2) rather than varied jointly with architecture: the final model must train under strong augmentation to simulate soundscape conditions, so searching architectures under light augmentation would converge on models optimised for clean focal audio that degrade at deployment. Phase 3 then performs targeted augmentation search with architecture frozen, reducing the joint search space from a multiplicative to an additive sequential search (O(arch) + O(aug)), that is, instead of testing every possible architecture-augmentation combination (multiplicative: 50 × 10 = 500 experiments), each dimension is searched independently and sequentially (additive: 50 + 10 = 60 experiments), because the architecture is already locked when augmentation search begins, making it tractable within a limited budget.

3.6 Critical Metric Decision: Macro-AP Over Macro-AUC

All local evaluations use the labelled train_soundscapes rather than a focal-audio held-out split, because focal-audio validation metrics are misleading: a model that overfits to clean focal audio can appear excellent locally while failing on real soundscapes. The choice of primary ranking metric was the result of a dedicated diagnostic study holding six exact submission models constant and comparing candidate local metrics against known Kaggle leaderboard outcomes:

Metric	Ranking Fidelity vs. Kaggle	Issue
Macro-AUC (overall)	Poor: inversions observed	Many rare classes have few positives in the local validation set, causing unstable per-class AUC estimates and misleading aggregate rankings
Macro Average Precision	Good: correct ranking preserved	Precision-recall balance; more sensitive to rare species discrimination
Median per-species AUC	Good: useful secondary indicator	Robust to outlier species but less effective than macro-AP
Focal-audio validation AUC	Very poor: misleading for soundscapes	Entirely different distribution from test data

Table 2: Local metric ranking fidelity vs. Kaggle public leaderboard.

We therefore adopted macro-AP on labelled soundscapes as the sole ranking metric for agent decisions and tournament selection. The critical property optimised is not absolute calibration but rank consistency. Macro-AP was the best available local proxy and corrected the most visible Perch/CNN inversion (Spearman $\rho = +0.61$ with Kaggle), though moderate correlation means close local scores should still be treated cautiously. An agent that optimises a misranking metric is worse than no agent at all.

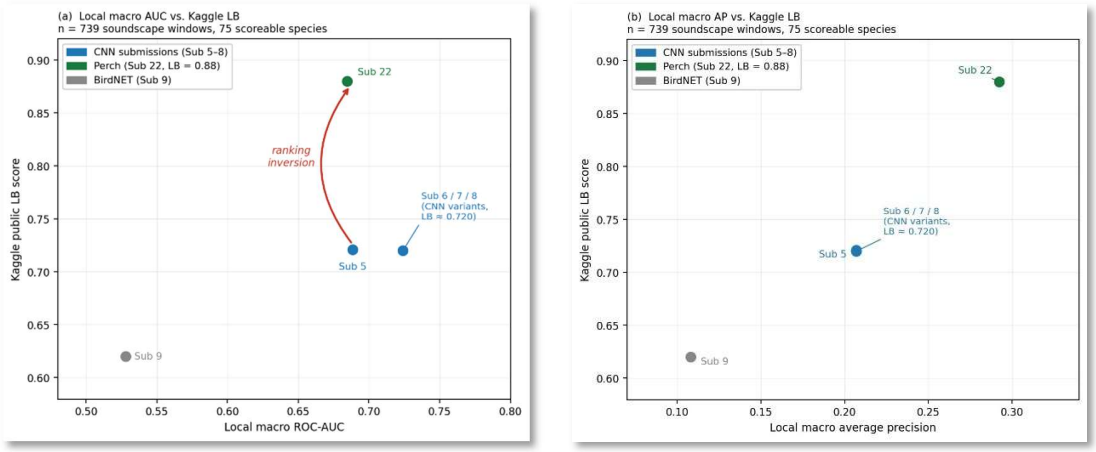


Figure 2: Metric diagnostic study results. (a) Local macro-AUC produces a ranking inversion: Perch scores lower locally than the CNN variants yet achieves a much higher Kaggle LB score. (b) Local macro-AP resolves this inversion, correctly ranking Perch highest on both axes, which indicates stronger alignment with the Kaggle leaderboard.

3.7 Pseudo-Labelling: Deliberately Non-Agentic

Stage 6 is kept outside agent control because evaluating different pseudo-labelling strategies would require re-running the full pipeline, which is a cost not justified by the marginal gains observed in our manual baseline. The absence of ground-truth species labels also makes reliable subset-level evaluation infeasible, as samples may be biased or systematically misclassified. and is too expensive to include in the agent's iterative loop. Fixed thresholds are used instead: primary species confidence 0.55, runner-up suppression 0.35. Although pseudo-labels are only assigned to high-confidence predictions, they are down-weighted during training so that pseudo-labelled samples contribute half as much as genuinely annotated examples.

4. Implementation Details

4.1 Repository Structure

- **configs/**: meta-agent pipeline parameters and stage configurations
- **src/**: all agent source code including Researcher, Coder, Executor, Evaluator, ExperimentMemory and PerchExperimentMemory modules with their embedded LLM prompt templates; CNN and Perch pipeline implementations; preprocessing utilities, augmentation presets, mel-spectrogram caching and embedding caching; SoundscapeEvaluator, SoundscapeEvalSuite and metric computation including macro-AP, AUC and median per-class AUC
- **logs/meta_agent/**: per-stage JSONL memories and experiment_timeline.json
- **notebooks/**: manual baseline notebooks for CNN, BirdNET and Perch

- **submission/**: Keras weights, head Python code, perch_inference.ipynb and CSV artifacts for Kaggle upload
- **README.md, requirements.txt**: setup and run instructions

4.2 Local LLM Setup

The agent interfaces with a locally hosted LLM via an OpenAI-compatible API (Ollama at localhost:11434). The Researcher uses a planning-capable model (e.g. qwen2.5-coder:7b); the Coder may use the same or a separate model. The agent is model-agnostic by design: swapping models requires only updating the endpoint configuration. All LLM calls are wrapped with retry logic and configurable timeout. Nevertheless, it is important to mention that some models will result in inefficient runs, such as deepseek-r1:8b, which often ended up in doom loops as a researcher.

4.3 Robustness Mechanisms

- **Repeated failures:** Short reasoning-string gets saved; Researcher prompt includes the five most recent failures to prevent re-proposing known-bad configurations.
- **Invalid code:** Coder output validated via AST parsing before subprocess launch, reducing time waste by syntax errors to a minimum.
- **JSON schema violations:** Researcher output validated against a JSON schema; fallback to default configuration on violation. A robust implementation tags fallbacks with an is_fallback flag so the Researcher can differentiate them in history summaries.
- **Memory/runtime:** Per-experiment configurable timeout and a 1,500-sample budget keeps wall-clock time bounded at each agent iteration.
- **Overengineering:** Bounded parametric search spaces such as CNN depth and epoch count are capped at defined maximum values, constraining the Coder to valid, testable configurations rather than unchecked open-ended generation that could cost a lot of time.

5. Experiment Log Analysis

5.1 CNN Track: Architecture Search (Submissions 18)

The CNN agent performed sequential, LLM-guided architecture search in the three search stages: 1a explored diverse CNN families via JSON specs and Coder-generated Keras models (guided by, not enumerating, the Appendix A hint space); 1b refined hypers within the winning architecture family; 1c held architecture fixed and searched augmentation only.

The winning architecture was a shallow CNN (depth=3, filters 32→64→128 with channel doubling, global average pooling, BN=True, dropout=0.3, lr=1×10^{-3}, Adam, weight_decay=1×10^{-4}), achieving macro_AP=0.087 and macro_AUC=0.573 on the labeled soundscape validation set. The CNN track was eliminated in the tournament against the Perch track (0.087 vs 0.285 macro_AP), consistent with the fundamental data-capacity mismatch discussed in Section 5.3.

5.2 Augmentation Ablations and Score Progression (Submissions 4-8)

After full-data retraining of R098 (Submission 4, Kaggle ≈0.65), augmentation ablations produced the most instructive results, all using identical architecture and hyperparameters:

Submission	Key Change	Kaggle AUC
v1.0-1.2	CNN baseline iterations	≈0.50–0.60
v1.3	Best CNN (R098), full-data training, 185 agent experiments	≈0.65
v1.4	+ Online augmentation: Gaussian noise + time shift + SpecAugment (no Mixup)	0.721 (+7.1 pp)
v1.5	+ Full augmentation + Mixup α=0.25	0.720 (+7.0 pp)
v1.6	+ Secondary-species labels as extra positives (weight 1.0)	≤0.721 (0 / −)
v1.7	+ Secondary-species labels (weight 0.5, down-weighted)	≤0.721 (0 / −)
v2.5	BirdNET agent: best head, full-data training + blend weight sweep	0.70
v3.1	Perch agent: best Phase 2 config; first full-data Perch submission	≈0.80+
v3.2	Perch agent (30 iter.) + extended LLM head search (residual + LayerNorm) + α=0.4 blend	0.88

Table 3: Full submission score progression (key versions shown). Complete history in Appendix D.

The 7.1-percentage-point gain from augmentation alone (v1.3 → v1.4) is a controlled experiment: both models use identical architecture, hyperparameters, and training data. The gain is entirely attributable to improved generalisation across the focal-to-soundscape domain gap. The negligible benefit of Mixup (v1.4 vs v1.5, Δ=0.001) reflects that macro-AUC is a rank-based metric less sensitive to probability calibration improvements. The failure of secondary labels (v1.6, v1.7) supports the hypothesis that secondary annotations carry a weaker and noisier supervision signal.

5.3 Transfer Learning Tracks

BirdNET (Sub. 11-14): BirdNET v2.4 maps audio windows to 1,024-dimensional embeddings. Embeddings were cached once and reused across all head experiments. The agent schedule progressed through depth exploration → width → regularisation → learning-rate sweep → batch-size sweep → augmentation preset search → embedding noise injection → prediction synthesis variants. Key findings: single-layer heads (512 hidden units) outperformed deeper MLPs early; a second layer with intermediate projection (512→256) improved the final Kaggle score; excessive depth (3+ layers) degraded performance; blend weight α in the range 0.2 to 0.5 consistently outperformed either head-only or BirdNET-only. Best standalone Kaggle submission: 0.70.

Perch (Sub. 15-22): Google Perch yields 1,536-d embeddings and also exposes zero-shot species logit scores over its pre-training vocabulary. The blending formula: $\hat{y} = \alpha \cdot \sigma(\text{Perch zero-shot logits}) + (1-\alpha) \cdot \sigma(\hat{y}_\text{head})$, where $\alpha=0.4$ was identified through systematic search. The Perch agent conducted 30 iterations. Best agent configuration at iteration 20 of submission version 3.2: 2-block MLP, hidden/proj dim 512, dropout 0.3/0.4, lr=5×10^{-4}, Adam, batch size 512, blend weight $\alpha=0.4$. Local macro-AP 0.294 translated to ≈0.88+ on Kaggle - this is the best fully autonomous agent result. A notable failure: 53% of Researcher outputs (16 of 30 iterations) did not conform to the JSON schema, triggering fallback to default configurations, wasting iteration budget and contaminating experiment history. The main reason for this was using the deepseek thinking model. You can see the full Perch head configuration in Appendix C. Following this exploration, Perch was identified as the stronger encoder and BirdNET was deactivated in favor of the Perch pipeline.

6. Manual Baseline Comparison

The manual notebook phase (detailed in Section 3.1) was a prerequisite for a functional agent, not an alternative to it. Table 5 compares outcomes across all model families and search strategies.

Approach	Type	Kaggle AUC	Notes
CNN from scratch (manual notebook)	Manual baseline	0.65	Best hand-tuned CNN, no systematic aug search
CNN from scratch (agent, R098)	Agent	0.65	Agent-identified optimal architecture
CNN from scratch (agent, aug v1.4)	Agent + full-data	0.72	+7.1 pp over manual CNN via augmentation search
BirdNET (manual baseline)	Manual baseline	0.62	First try with a pretrained model
BirdNET (agent, best head)	Agent	0.70	Agent-driven head search on 1,024-d embeddings
Perch (manual baseline)	Manual baseline	0.79	Established Perch dominance; triggered metric study
Perch agent (iter. 20)	Agent	0.80+	Outperformed manual baseline through smart agent experiments
Perch + extended LLM head search	Agent + manual extension	0.88	Final submission; agent code optimized with small tweaks

Table 4: Manual baselines vs. agent outputs across all model families.

Two comparisons stand out. First, the agent's CNN track (0.72) outperformed the equivalent manual CNN baseline (0.65) by approximately 7 percentage points, a gain attributable to systematic augmentation and hyperparameter search across 185 experiments, a scale of search impractical to conduct manually. Second, the Perch agent efficiently identified the right encoder family, blend strategy, and head architecture, with the gap between the manual baseline (0.79) and final submission (0.88) reflecting the complementary value agent-driven search on top of human knowledge.

7. Analysis & Discussion

7.1 The Representational Bottleneck of CNN-from-Scratch

The most striking observation in the experimental record is the hard ceiling for CNN models trained from scratch, even after exhaustive search across configurations spanning many dimensions (0.721 Kaggle LB with augmentation). We attribute this ceiling to a fundamental data-quantity/model-capacity mismatch: a CNN must simultaneously learn a general-purpose acoustic feature extractor and a species-level classifier across 234 classes with severe imbalance, but with 25 species having few training examples and a median of only 126 recordings per species, there is insufficient data for the convolutional layers to reliably learn discriminative filters for rare species, precisely those weighted equally in macro-AUC.

Pre-trained acoustic encoders resolve this bottleneck by separating the two objectives. BirdNET and Perch have been trained on corpora orders of magnitude larger than the BirdCLEF training set; their feature spaces already encode bird vocalization structure at an abstraction level exploitable by a shallow head with far fewer labelled examples. The empirical validation is clear: Perch (≈0.88) >> BirdNET (≈0.70) >> from-scratch CNN (0.721 Kaggle LB), confirming that domain match between pre-training corpus and target task is the dominant factor.

7.2 LLM-Guided Search: What Works, What Fails

The Researcher LLM demonstrates clear, reproducible value when the experiment history contains sufficient signal: it correctly identifies that moderate learning rates ($\approx 10^{-3}$) consistently outperform extremes; that residual connections improve gradient flow in deeper heads; that blend weights around 0.3 perform well; and that layer normalisation benefits transformer-style heads.

Two structural failure modes were observed. Failure mode 1: JSON parse failures and fallback contamination: approximately 53% of Perch agent Researcher outputs did not produce valid JSON (16 out of 30 logged iterations), triggering fallback to default configurations. These were logged as experiments, wasting iteration budget. A robust implementation requires JSON validation before passing to the Coder, retry with a stricter prompt on validation failure and explicit is_fallback tagging in the memory. Failure mode 2: Bounded search space as a binding constraint: when the proposed architecture lies outside the pre-defined boundaries, the agent uses predefined fallback values.

7.3 Augmentation as Domain Adaptation

The 7.1-percentage-point improvement from augmentation alone (v1.3 → v1.4, 0.65 → 0.721 Kaggle AUC) is a controlled experiment: identical architecture and data, only augmentation added. This confirms that generalisation failure in the CNN track is driven by distribution shift, not architectural underfitting. The augmentation design rationale and preset definitions are detailed in Section 2.4.

7.4 Zero-Shot Blending as an Implicit Ensemble

The zero-shot logit blending formula $\hat{y} = 0.4\ \sigma(\text{Perch zero-shot logits}) + 0.6 \cdot \sigma(\hat{y}_\text{head})$ implicitly ensembles two qualitatively different classifiers with complementary error profiles. The trained head is optimised on the BirdCLEF 2026 species set and strong on common species; the Perch zero-shot signal provides a prior based on acoustic similarity to the pre-training vocabulary, particularly informative for rare species with scarce training data. Setting $\alpha=0$ loses the zero-shot prior for rare species; setting $\alpha=1$ loses task-specific supervision for common species. Our validated $\alpha=0.4$ exploits the asymmetric strengths of both components and was part of the best autonomous configuration.

8. Reflections on Course Content

The project traversed the full range of deep learning techniques from the Advanced Predictive Analytics course, providing systematic empirical evaluation of each in a real-world competition context:

Technique	Application in This Project	Effectiveness & Key Insight
Signal preprocessing (spectrograms)	Mel-spectrogram pipeline: 32 kHz, 64 mels, 128 frames, log-scaled, Validated against alternatives.	Foundational. Configuration fixed after empirical validation across three alternatives. BirdNET uses its own internal front-end at 48 kHz.
2D CNNs on spectrograms	Full CNN-from-scratch track: Conv2D, BN, global average pooling, residual connections. 185 experiments, 14 dimensions.	Ceiling ≈0.65 AUC. Limited by data-to-capacity mismatch for 234-class problem with many rare species. Residuals and global avg pooling consistently beneficial.
Transfer learning	Two pre-trained encoders: BirdNET v2.4 (bird-specific audio, 1,024-d) and Google Perch (large-scale audio, 1,536-d). Frozen backbone + trained head.	Primary performance driver. Domain match critical: Perch (0.88) >> BirdNET (0.70) >> from-scratch (0.65). Pre-training corpus size and domain relevance both matter.

Technique	Application in This Project	Effectiveness & Key Insight
Data augmentation	Online spectrogram augmentation: Gaussian noise, time-axis shift, SpecAugment time/frequency masking, Mixup. Applied probabilistically at batch time.	+7.1 pp controlled gain (v1.3→v1.4). Directly bridges focal-to-soundscape gap. Mixup adds negligible benefit under macro-AUC.
Multi-label classification	234-species sigmoid head; binary cross-entropy loss; per species AUC evaluation	Frequency weighting essential: without it, the model collapses to predicting only common species.
Ensemble methods	Fixed linear blending of Perch zero-shot logits and trained head outputs (α=0.4). Also implicit ensemble across training data types (focal + soundscapes + pseudo-labelled).	Critical technique: zero-shot blending with α=0.4 was part of the autonomous Perch config (≈0.80 Kaggle). The final +8 pp to 0.88 came from the LLM-guided extension to residual MLP architectures; blending and architecture together drive the system.
Hyperparameter optimisation	LLM-guided search across LR, WD, batch size, dropout, head depth/width, guided by experiment history rather than random search.	Agent consistently identifies low LR (≈10^{-4}), moderate dropout (0.3–0.4), Adam optimiser as optimal. SGD with momentum decisively inferior (tested in Perch iteration 8, scored 0.631).

Table 5: Course techniques applied, with effectiveness and key insights.

9. Limitations & Future Work

- **Agent search space as a hard ceiling on architectural novelty:** Agent search is bounded by its available tools: while the agent can effectively explore CNN architectures and pretrained model families such as BirdNET and Perch, it cannot propose entirely new directions it lacks tooling for. For example, integrating an alternative pretrained model such as YAMNet would require extending the toolkit first. Future designs should support dynamic tool expansion, allowing the agent to propose and test new model families beyond its pre-defined set.
- **Fixed spectrogram configuration:** Only three mel/frame combinations were evaluated. A broader search over filter counts, hop lengths, window sizes, and frequency range limits may identify superior configurations. Raw waveform encoders (wav2vec 2.0, AudioMAE) bypassing the mel bottleneck entirely were not explored.
- **No temporal modelling across clips:** Each 5-second clip is classified independently. Species presence in soundscapes is temporally autocorrelated: a species detected in one window is more likely present in adjacent windows. LSTM/GRU, transformer-based temporal aggregation, or sliding-window ensembling could exploit this structure.
- **Fixed scalar blend weight:** The final α=0.4 applies uniformly to all 234 species. A per-species meta-learner trained on validation soundscapes to assign per-species blend weights could substantially improve performance, especially for species where the fixed weight is suboptimal.
- **Unsuccessful secondary label integration:** Submissions v1.6 and v1.7 failed to improve upon primary-only training. More principled approaches such as treating secondaries as a regularisation term with a lower loss weight; curriculum filtering; or using secondaries only for species with very few primary examples , warrant further investigation.
- **LLM output robustness:** The model frequently produces erroneous outputs in both generated code and experiment suggestions, posing a reliability risk. By switching to a more reliable model such as qwen2.5-coder:7b we were able to reduce failures to a minimum, though the system should ideally adapt its prompting strategies dynamically based on the selected model, as different models exhibit different behaviours across the Coder and Researcher roles.
- **No post-hoc cross-family ensemble:** The final submission is a single Perch-based system. An ensemble across BirdNET (0.70) and the Perch final model was not explored due to compute constraints and the decision to focus on the stronger Perch encoder.
- **Local metric calibration:** While macro-AP reliably preserves ranking, absolute values differ substantially from Kaggle scores (e.g. 0.294 local for a model scoring ≈0.88 on Kaggle). A calibration study relating local macro-AP to Kaggle AUC per model family would improve decision criteria further.

10. Individual Contributions

Member	Role	Primary Contributions
Enrico Zafiris	Team Lead	Overall architecture, experimental strategy and guidance; development of CNN and Perch baseline notebooks as reproducible starting points; orchestration and integration of all parallel workstreams; LLM-guided extended head architecture search beyond the 30-iteration Perch

Member	Role	Primary Contributions
		agent parametric space; production of the final 0.88 Perch-based zero-shot/head blended submission.
Lasse Aaron Ortmann	EDA, CNN Track & multi-label classification	Conducted the Exploratory Data Analysis; drove the early CNN-from-scratch experimental track; added the multi label classification set up with sigmoid outputs and binary cross-entropy; established the data pipeline, mel-spectrogram caching, and agent search infrastructure that all subsequent tracks built upon.
Moritz Üblacker	Perch Track	Investigated Google Perch acoustic encoder; adapted it to the BirdCLEF 2026 species set via genus-level proxy mapping; built the Perch-based agent and PerchExperimentMemory module; conducted 30 iterative agent experiments; designed and implemented the data augmentation pipeline (augmentation.py), including the light, medium, and high presets; experiments confirmed the high preset as optimal for all agent search phases.
Alexander Wanstrath	BirdNET Track	Investigated BirdNET v2.4 as a transfer learning encoder; conducted systematic manual head search covering depth, regularisation, LR, and augmentation; produced a standalone Kaggle submission scoring 0.70; findings informed the decision to focus the autonomous pipeline on Perch as the stronger acoustic encoder.

Table 6: Team member roles and primary contributions.

11. Conclusion

This project demonstrates that the combination of domain-specific pre-trained acoustic encoders, LLM-guided autonomous head optimisation, principled metric selection, and zero-shot logit blending can achieve competitive performance on a challenging real-world PAM task. Starting from a naïve convolutional baseline of ≈0.65 macro-AUC, our agent pipeline, spanning 22 submission versions and more than 215 tracked agent iterations, achieved a final public leaderboard score of 0.88, a 38-pp improvement that substantially outperforms all hand-tuned manual baselines.

The most important lessons are methodological: (1) representation quality matters more than architectural search; (2) metric choice is not an agent-level decision. Discovering that macro-AUC inverted the Perch/CNN ranking, and switching to macro-AP, was the single most consequential diagnostic finding; (3) staged pipeline design enables principled compute allocation; (4) augmentation is domain adaptation, not regularisation; (5) zero-shot blending is an effective rare-species ensemble strategy; (6) agent search and expert design are complementary: human-designed baselines define the tools and search space (≈0.80 AUC), with the agent's exploration driving the final +8 pp gain to 0.88.

Beyond reusability, the results confirm a broader principle: an agent that autonomously explores, adapts based on its own experimental history, and iterates beyond human-designed baselines can close performance gaps that expert tuning alone cannot

12. References & Links

- **GitHub Repository:** https://github.com/enricozafcode/APA-MUE-LO-AW-EZ
- **Video Presentation:** https://www.youtube.com/watch?v=ZbqlASTRl54
- **Kaggle Competition:** https://www.kaggle.com/competitions/birdclef-2026
- **Kaggle Team Submissions:** We don't breathe, we code

Chollet, F. Deep Learning with Python, 2nd ed., Manning, 2021.

Kaplan, J. et al. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020.

Hoffmann, J. et al. Training Compute-Optimal Large Language Models. arXiv:2203.15556, 2022.

Kahl, S. et al. BirdNET: A deep learning solution for avian diversity monitoring. Ecological Informatics 61, 2021. Hamer, H. et al. Google Bird Vocalization Classifier (Perch): A Large-Scale Acoustic Model for Bird Species Recognition. arXiv:2309.05927, 2023.

Park, D.S. et al. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. Interspeech, 2019.

Zhang, H. et al. mixup: Beyond Empirical Risk Minimization. ICLR, 2018.

Appendix

Appendix A: Hint Space

Full 14-dimensional search space explored by the CNN agent across 13 experiments:

Dimension	Range / Options
Depth (conv blocks)	2, 3, 4, 5, 6, 8
Filter base	16, 32, 64, 128
Learning rate	1e-2, 5e-3, 1e-3, 5e-4, 1e-4
Weight decay	0.0, 1e-4, 1e-3
Classifier hidden units	0, 128, 256, 512
Residual connections	True / False
Batch normalisation	True / False
Pooling type	global_avg, global_max, flatten
Dropout rate	0.0, 0.2, 0.3, 0.5
Batch size	16, 32, 64
n_mels / n_frames	64 or 128 / 128 or 256

Appendix Table A1: CNN agent search space (14 dimensions, 13 experiments completed).

Appendix B: CNN Winning Configuration (R098, v1.4)

Parameter	Value / Notes
Depth (conv blocks)	8; channel doubling from base 32
Batch normalisation	True (consistently beneficial across all search phases)
Residual connections	True (improves gradient flow in >6-block architectures; marginal for ≤4 blocks)
Pooling	Global average pooling (outperformed global max pooling in focused search)
Classifier hidden units	0 (direct logits to 234 classes; intermediate layers degraded performance)
Dropout	0.0 (batch normalisation provides sufficient regularisation)
Optimizer	Adam, lr = 1.78×10^{-4}, wd = 1.06×10^{-5}
Batch size	32 (slightly preferred over 64/128 in focused search)
n_mels / n_frames	64 / 128 (empirically validated; 128 mels/256 frames increased training time without consistent gain)
Augmentation (v1.4)	Gaussian noise σ=0.015 + time shift 10% + time/freq masking 8% (no Mixup)
Local macro-AUC (search)	0.637 (best R098 result in cheap search; full-data improves to ≈0.65)
Kaggle Public AUC (augmented)	0.721 (v1.4); Mixup (v1.5) gives 0.720, effectively identical

Appendix Table B1: CNN winning configuration (R098) with augmentation v1.4.

Appendix C: Perch Head Winning Configuration (Iteration 20)

Parameter	Value / Notes
Encoder	Google Perch (ONNX, 1,536-d, frozen)
Species mapping	Genus-level proxy mapping to 234 BirdCLEF 2026 species
MLP blocks (n_blocks)	2 sequential projection-dropout-activation blocks
hidden_dim / proj_dim	512 / 512
dropout_block	0.3 (applied after activation in each block)
dropout_final	0.4 (applied to final representation before output layer)
Optimizer	Adam, lr = 5×10^{-4}; SGD with momentum decisively inferior (tested in iter. 8: scored 0.631)

Parameter	Value / Notes
Batch size	512 (stable for embedding-based training; 128 and 256 also competitive)
Epochs	25 with early stopping patience 5 on soundscape macro-AP
Perch blend weight α	0.4 (40% zero-shot Perch, 60% trained head; identified through sweep in iter. 20)
Local macro-AUC (agent)	0.6847 (best agent result; final v3.2 adds residual+LayerNorm: 0.88)

Appendix Table C1: Perch agent best head configuration (iteration 20).

Appendix D: Full Submission Version History

Version	Folder	Key Change / Description	Kaggle AUC
v1.0	sub_01	Initial CNN baseline, default configuration, no augmentation	0.651
v1.1	sub_02	CNN with Phase 1 linear scan improvements	0.500
v1.2	sub_03	CNN with Phase 2A random search improvements	0.652
v1.3	sub_04	Best CNN (R098), full-data training, 185 CNN agent experiments	0.650
v1.4	sub_05	+ Online augmentation: Gaussian noise + time shift + SpecAugment (no Mixup)	0.721
v1.5	sub_06	+ Full augmentation with Mixup α=0.25 added	0.720
v1.6	sub_07	+ Secondary-species labels as additional positives (weight 1.0)	0.720
v1.7	sub_08	+ Secondary-species labels (weight 0.5, down-weighted)	0.720
v2.0	sub_09	MobileNetV2 ablation (planned but excluded from final pipeline)	0.620
v2.1	sub_10	BirdNET v2.4 baseline notebook, initial head exploration	0.621
v2.2	sub_11	BirdNET agent: depth exploration (Phase 1–3)	0.701
v2.3	sub_12	BirdNET agent: regularisation + blend weight search (Phase 4–6)	0.735
v2.4	sub_13	BirdNET agent: augmentation preset search + embedding noise (Phase 7–9)	0.748
v2.5	sub_14	BirdNET agent: best head, full-data training + blend α sweep	0.758
v2.6	sub_15	Perch baseline notebook; embedding caching; initial single-layer head	0.793
v2.7	sub_16	Perch agent: Phase 2 architecture exploration (iter. 1–10)	0.793
v2.8	sub_17	Perch agent: Phase 2 continued + refinement starts (iter. 11–20)	0.793
v2.9	sub_18/19	Perch agent: Stage 2 refinement continued; ProtoSSM variant exploration	0.895
v3.0	sub_20	Perch agent: Phase 2 augmentation search; embedding noise variants	0.761
v3.1	sub_21	BirdNet agent: First Agent configuration	0.704
v3.2	sub_22	Perch agent (30 iter.) + extended LLM head search (residual + LayerNorm)	0.880

Appendix Table D1: Complete submission version history (all 22 versions).


v 3.5 Another Perch Winner - Version 1
Succeeded · Enrico777 · 2d ago · Notebook v 3.5 Another Perch Winner | Version 1
0.868
v 3.4 Perch Agent deepseek-r1:8b qwen2.5-coder:7b - Version 2
Succeeded · Enrico777 · 3d ago · Notebook v 3.4 Perch Agent deepseek-r1:8b qwen2.5-coder:7b | Version 2
0.869
v 3.3 Perch Agent deepseek research / gemma4 coder - Version 5
Succeeded · Moritz Üblacker · 3d ago
0.804
v 3.2 Perch Agent deepseek-r1:8b qwen2.5-coder:7b - Version 1
Succeeded · Enrico777 · 5d ago · Notebook v 3.2 Perch Agent deepseek-r1:8b qwen2.5-coder:7b | Version 1
0.880
notebook4b15b27022 - Version 11
Succeeded · Alexander Wanstrath · 7d ago · v 3.1 BirdNet
0.704
v 3.0 Perch - Version 3
Succeeded · Moritz Üblacker · 9d ago
0.761
v 2.9 ProtoSSM - Version 1
Succeeded · Enrico777 · 15d ago · Notebook v 2.9 ProtoSSM | Version 1
0.895
v 2.8 Perch Pseudo - Version 2
Succeeded · Enrico777 · 16d ago · Notebook v 2.8 Perch Pseudo | Version 2
0.793
v 2.7 Perch Pseudo - Version 1
Succeeded · Enrico777 · 16d ago · Notebook v 2.7 Perch Pseudo | Version 1
0.793
v 2.6 Perch Try - Version 4
Succeeded · Enrico777 · 16d ago · Notebook v 2.6 Perch Try | Version 4
0.793
2.5 BirdNet Pseudo Sequence Ensemble Head - Version 1
Succeeded · Enrico777 · 17d ago · 2.5 BirdNet Pseudo Sequence Ensemble Head | Version 1
0.758
v 2.4 BirdNet Pseudo Sequence Res Head - Version 5
Succeeded · Enrico777 · 17d ago · v 2.4 BirdNet Pseudo Sequence Res Head | Version 5
0.748
v 2.3 BirdNet Pseudo Labeling Sequence Label - Version 4
Succeeded · Enrico777 · 17d ago · Notebook v 2.3 BirdNet Pseudo Sequence Label | Version 4
0.735

v 2.2 BirdNet Pseudo Labeling - Version 3 Succeeded · Enrico777 · 18d ago · v 2.2 BirdNet Pseudo Labeling | Version 3	0.701	☐
v 2.1 BirdNet Test - Version 2 Succeeded · Enrico777 · 18d ago · v 2.1 BirdNet Test | Version 2	0.692	☐
v 2.0 First Pretrained Model - Version 2 Succeeded · Enrico777 · 20d ago · Notebook v 2.0 First Pretrained Model | Version 2	0.620	☐
v 2.0 First Pretrained Model - Version 1 Notebook Threw Exception · Enrico777 · 20d ago · Notebook v 2.0 First Pretrained Model | Version 1		
v 1.7 same as 1.6 w secondary label (w = 0.5) - Version 2 Succeeded · Enrico777 · 20d ago · Notebook v 1.7 same as 1.6 w secondary label (w = 0.5) | Version 2	0.720	☐
v 1.6 same as 1.4 w secondary label (w = 1) - Version 1 Succeeded · Enrico777 · 20d ago · Notebook v 1.6 same as 1.4 w secondary label (w = 1) | Version 1	0.720	☐
v 1.5 (same as 1.3 w basic data augmentation) - Version 1 Succeeded · Enrico777 · 21d ago · Notebook v 1.5 (same as 1.3 w basic data augmentation) | Version 1	0.720	☐
v 1.4 (same as 1.3 2 data augmentation) - Version 1 Succeeded · Enrico777 · 21d ago · Notebook v 1.4 (same as 1.3 2 data augmentation) | Version 1	0.721	☐
v 1.1. Linear and Random Experiments - Version 5 Succeeded · Enrico777 · 21d ago · Notebook v 1.1. Linear and Random Experiments | Resub bc of sub mistake	0.652	☐
v 1.3 Randomized Experiments scaled (>12h) - Version 1 Succeeded · Enrico777 · 21d ago · Notebook v 1.3 Randomized Experiments scaled (>12h) | Version 1	0.650	☐
v 1.2 Linear and Randomized Experiments Test/Train - Version 5 Succeeded · Enrico777 · 22d ago · Notebook v 1.2 Linear and Randomized Experiments Test/Train | Version 5	0.652	☐
v 1.2 Linear and Randomized Experiments Test/Train - Version 3 Succeeded · Enrico777 · 22d ago · Notebook v 1.2 Linear and Randomized Experiments Test/Train | Version 3	0.500	☐
v 1.1. Linear and Random Experiments - Version 4 Succeeded · Enrico777 · 23d ago	0.500	☐
Test_One - Version 1 Succeeded · Enrico777 · 1mo ago · Notebook First Iteration | Version 1	0.651	☐

Appendix E: Improvement of the Agent within one Loop

